



Trial Activation

Data Analyst Community Challenge

Hosted by Splendor Analytics

The Problem

At Splendor Analytics, we run a 30-day free trial for new organisations signing up to our workforce management platform. The platform covers everything from shift scheduling and time tracking to payroll approvals and team communications.

The challenge is this: we do not know what a "good" trial looks like. Our product team can see that roughly 1 in 5 trialists eventually converts to a paying customer, but they cannot tell who is on track to convert, when to intervene, or which features actually matter to the decision. Without that clarity, every onboarding improvement is a guess.

To fix this, we want to define what Trial Activation means for our product — a specific set of in-app behaviours that signal a trialist has genuinely experienced our core value. We then want to build the data infrastructure to track activation at scale, and run the analysis to understand whether our current trialists are reaching it.

Your job is to do exactly that: dig into the raw behavioural data, find what matters, define the goals, and build the models.

The Dataset

You will be provided with a raw CSV dataset of behavioural events from organisations that started their trial between January and March of last year. The dataset has the following schema:

Column	Type	Description
organization_id	string	Unique identifier for each trialling organisation
activity_name	string	Name of the product activity performed
timestamp	datetime	When the activity occurred
converted	boolean	Whether the organisation converted to paid
converted_at	datetime	Timestamp of conversion (NaT if not converted)
trial_start	datetime	When the trial started
trial_end	datetime	Trial expiry date (trial_start + 30 days)

A full description of all activity_name values is provided in Attachment 1 at the end of this document.

The Tasks

Complete all three tasks below and submit your full solution as a public git repository.

1. **Explore and clean the dataset using Python. Using advanced analytics practices, discover which activities are indicative of conversion and define trial goals. Describe your approach.**



2. **Based on your analysis, build an SQL-based model providing two sources that would live in the marts layer of the data warehouse:**
 - Trial Goals — tracks whether each trialist organisation has completed each of the trial goals you defined.
 - Trial Activation — tracks which organisations have fully completed all trial goals, thereby achieving "Trial Activation".
3. **Run basic descriptive analyses using Python to inform the product team's decision-making. Also consider commonly used product metrics you can derive from the data.**

Please provide your solution in a git repository with your full code.

How to Submit

- Make your repository public on GitHub or GitLab before submitting.
- Ensure your repo includes a README.md, a requirements.txt, Python notebooks or scripts for Tasks 1 and 3 (with outputs rendered), and SQL files for Task 2.
- Submit via the Google Form linked in the post — paste your public repository URL and your Twitter/X handle into the form.
- One submission per person. If you submit more than once, only your most recent form entry before the deadline will be counted.

Deadline: 4 days from the date this post is published.

Grading Rubric

All valid submissions will be evaluated against the rubric below. Scores are final. The highest-scoring submission wins the full ~~N~~100,000 prize.

Criterion	Marks	What Judges Will Look For
1. Data Cleaning & Exploration	20	Proper deduplication; timestamp parsing; derived fields; documented quality checks; meaningful EDA visualisations.
2. Conversion Driver Analysis	20	Multiple rigorous methods (statistical tests, ML models, engagement segmentation); correct interpretation; honest findings even if null.
3. Trial Goal Definition	20	Goals grounded in product-value logic with supporting evidence; realistic completion rates; acknowledged as hypotheses not guaranteed levers.
4. SQL Model Quality	15	Correct, optimised SQL; accurate grain for both mart tables; staging layer present; dbt conventions or equivalent documentation.
5. Descriptive Analytics & Product Metrics	15	Correct product metrics (conversion rate, retention, module adoption, time-to-convert); actionable recommendations clearly communicated.
6. Repo Structure & Code Quality	10	Clean git history; logical folder structure; README; requirements.txt; readable, commented code; no hardcoded data or credentials.



TOTAL	100	<i>Highest overall score wins. Ties broken by score on criterion 2.</i>
--------------	------------	---

Score	Grade	Interpretation
85–100	Distinction	Exceptional — strong contender for the prize
70–84	Merit	Solid professional-grade submission
55–69	Pass	Competent but missing key insights or polish
Below 55	Fail	Does not meet the minimum standard

Automatic Disqualification

Any submission meeting one or more of the following criteria will be immediately disqualified, regardless of analytical quality:

- Plagiarised or AI-generated analysis presented as original work
- Missing or empty git repository link in the comments
- Code that does not run end-to-end without manual intervention
- No README or zero commit history
- Submission posted after the deadline

Terms & Conditions

- Open to all data professionals and students worldwide.
- One submission per person. Duplicate accounts will be disqualified.
- The prize (~~£~~\$100,000) will be transferred to the winner within 7 business days of announcement.
- Splendor Analytics reserves the right to share and feature winning submissions for educational purposes, with full attribution to the author.
- The judge's decision is final and binding.
- Any form of cheating, collusion, or plagiarism results in permanent disqualification.

Good luck. Let your data do the talking.

Attachment 1: Activity Name Descriptions

Activity Name	Description
Scheduling.Availability.Set	Setting availability for scheduling
Scheduling.Shift.Created	Creating new shifts
Scheduling.Shift.AssignmentChanged	Last-minute shift assignment changes
Scheduling.Template.ApplyModal.Applied	Applying shift templates
Scheduling.ShiftSwap.Created	Requesting to swap shifts
Scheduling.ShiftSwap.Accepted	Accepting a shift swap (employee or admin)
Scheduling.ShiftHandover.Created	Requesting to hand over shifts
Scheduling.ShiftHandover.Accepted	Accepting a shift handover (employee or admin)



Scheduling.OpenShiftRequest.Created	Requesting open shifts
Scheduling.OpenShiftRequest.Approved	Approving open shift requests (admin)
Mobile.Schedule.Loaded	Viewing the overall schedule
Shift.View.Opened / ShiftDetails.View.Opened	Viewing details of specific shifts
Absence.Request.Created	Creating absence requests
Absence.Request.Approved / Rejected	Approving or rejecting absence requests
PunchClock.PunchedIn / PunchedOut	Clocking in/out via the punch clock
Break.Activate.Started / Finished	Starting or ending a break period
PunchClockStartNote.Add.Completed	Adding a note when punching in
PunchClockEndNote.Add.Completed	Adding a note when punching out
PunchClock.Entry.Edited	Editing time clock entries
Scheduling.Shift.Approved	Approving shifts for payroll
Timesheets.BulkApprove.Confirmed	Approving timesheets
Integration.Xero.PayrollExport.Synced	Syncing payroll with external systems
Revenue.Budgets.Created	Creating manual budget entries or payroll budget %
Communication.Message.Created	Sending team communications